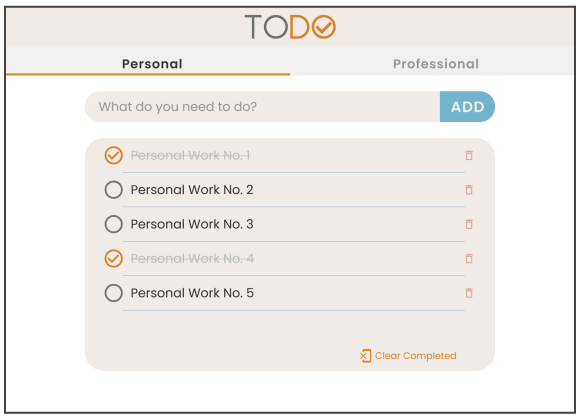
HW4: ReactJS Implementation of a Todo UI

Topic: React, Component Structure, and Styling
Tools: Vite, React, CSS or TailwindCSS

Overview

In this homework, you will implement the user interfaces you designed in HW3 using React. Your task is to translate your Figma designs into working React applications, focusing on component structure, layout, and styling. This assignment emphasizes the transition from visual design to code, a core skill in modern frontend development.



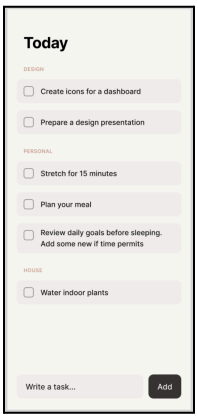


Fig. 1: Design Screenshots: Use the following screenshots to create corresponding Figma designs.

You will create two separate React projects, one corresponding to the desktop screen and one corresponding to the mobile screen from HW3. Each project should faithfully reproduce the visual layout and structure of its respective design.

At this stage, the emphasis is on UI implementation, not application completeness. Your apps do not need full task persistence or advanced features. Instead, the goal is to practice structuring React components and applying styles to match an existing design specification.

Project Setup

You are required to create two independent projects, either using Vite+React or React-Native+Expo. One project should implement the desktop version of the todo application, and the other should implement the mobile version. These must be separate projects rather

than different views within a single project, as this helps reinforce how React applications are structured and configured from scratch.

You may use JavaScript (JSX) or TypeScript (TSX). Using TSX is encouraged and will be awarded bonus points, as it reflects industry-standard practices and stronger type discipline. For styling, you are free to choose either vanilla CSS, TailwindCSS, or Stylesheets. Regardless of which approach you choose, your styling should be intentional, readable, and consistent with the design you created in HW3.

Creating a Vite + React Project

For this assignment, you will use **Vite** to scaffold your React applications. Vite is a modern frontend build tool that provides fast development startup and a simple project structure. Before you begin, make sure you have a recent version of **Node.js** installed on your system. If running `node -v` in your terminal prints a version number, you are ready to proceed.

Creating a React Project Using JSX

If you choose to implement your project using standard JavaScript with JSX, you should create a Vite project using the React template. In your terminal, navigate to the directory where you would like your project to live, then run the following command.

```
npm create vite@latest todo-desktop -- --template react
```

After the project is created, navigate into the project directory and install the required dependencies.

```
cd todo-desktop
npm install
```

Once installation is complete, you can start the development server by running the following command.

```
npm run dev
```

This will start a local development server and print a URL in the terminal. Opening that URL in your browser will display your running React application. You can now begin modifying the files in the `src` directory to implement your UI.

Creating a React Project Using TypeScript (TSX)

If you choose to implement your project using TypeScript, you should create a Vite project using the React + TypeScript template. This option is encouraged and is eligible for bonus points on

this assignment. As with the JSX version, begin by navigating to your desired project directory and running the following command.

```
npm create vite@latest todo-mobile -- --template react-ts
```

After the project is created, move into the project directory and install dependencies.

```
cd todo-mobile
npm install
```

You can then start the development server in the same way.

```
npm run dev
```

When using the TypeScript template, you will notice that React components use the `.tsx` file extension instead of `.jsx`. You are encouraged to add appropriate type annotations for component props and state, but you do not need to introduce advanced TypeScript features beyond what is necessary for this assignment.

Notes and Expectations

Each project should be created as its **own independent Vite application**. For this homework, you should create two separate projects, one for the desktop UI and one for the mobile UI, even if they share similar structure or components. This is intentional and helps reinforce familiarity with React project setup and configuration.

If you encounter errors during setup, verify that your Node.js version is up to date and that you ran `npm install` before starting the development server. You are encouraged to ask questions early if setup issues arise, as project configuration problems can block progress on the rest of the assignment.

Implementation Expectations

Your React applications should reflect the visual structure of your Figma designs as closely as possible. This includes layout, spacing, typography hierarchy, and clear visual differentiation between elements such as completed and incomplete tasks. Minor differences in exact colors or fonts are acceptable; however, the overall structure, hierarchy, and visual intent of the design should be clearly recognizable when comparing the implementation to your Figma file.

In addition to visual accuracy, your applications are expected to be functionally interactive at the front end. You should manage application state using React's `useState` hook and ensure that user actions meaningfully affect the rendered UI. At a minimum, this includes:

1. Maintaining a list of tasks in state

2. Allowing new tasks to be added through the input interface
3. Updating task state to reflect completed and incomplete statuses.
4. Changes to state should immediately and correctly update the rendered components.

Scope Control

To keep the focus on UI implementation, you are not required to implement data persistence, routing, authentication, or backend communication. You also do not need to support multiple pages or dynamic screen resizing beyond what is necessary to reasonably display your chosen desktop or mobile layout.

This assignment is not about building a production-ready todo app. It is about accurately implementing a given design using React and modern frontend tooling.

Submission Instructions

You will submit a single repository or archive containing both projects, clearly labeled to distinguish the desktop and mobile implementations. Each project should be runnable independently using standard Vite commands.

Your submission must include a short note indicating whether you used JSX or TSX, as well as whether you used vanilla CSS or TailwindCSS. This information can be included in a short section at the top of your repository README or in a separate text file.

Make sure your projects build and run without errors. Incomplete or non-running projects will receive significant deductions.

Grading Criteria

Your work will be evaluated based on the following criteria, which reflect both the accuracy of your implementation and the quality of your React code. The base assignment is graded out of 100 points, with additional bonus points available.

Criterion	Description	Points
Design Fidelity	The React implementation closely matches the corresponding HW3 Figma designs in terms of layout, spacing, visual hierarchy, and differentiation between UI elements such as completed and incomplete tasks.	30

Functional React State	The application correctly uses React state (`useState`) to manage tasks, including adding new tasks and updating completed or incomplete status, with UI updates responding appropriately to user interaction.	25
Component Structure	The UI is decomposed into clear, logical React components that reflect meaningful interface boundaries rather than being implemented as a single monolithic component.	20
Styling Quality	Styling is applied cleanly and consistently using either vanilla CSS or TailwindCSS, with readable class usage and intentional organization.	15
Project Setup & Code Quality	The Vite project is correctly configured, runs without errors, and demonstrates readable, well-organized code with appropriate naming and file structure.	10
Bonus: TypeScript (TSX)	The project is implemented using TypeScript (TSX) rather than JSX, with appropriate typing of props and state.	**+10**

Total: 100 points + up to 10 bonus points

Looking Ahead

This assignment completes the design-to-implementation pipeline that began in HW3. In future work, you will extend these skills toward more dynamic behavior and full-stack integration. The React applications you build here will serve as a strong foundation for that next step.